

Creative Writing Thesaurus & NLP Pipeline

Elliot Klauke

Keywords: Python, Gensim, Hugging Face, NLTK.

1 Purpose

As a creative writer, I have always found myself searching for synonyms better than a word I had in mind. A traditional thesaurus fulfills this need to some extent, but I wanted to develop a thesaurus geared towards the language of classical literature, including words that have gone largely extinct in contemporary writing.

In addition to this, I wanted to explore the potential for natural language processing and machine learning to find contextually similar words for a lightweight, offline application that I could easily access as I wrote.

2 Data and Methods

The model uses the Gensim Word2Vec library and trains with 5 epochs using sedthh's Project Gutenberg database's full 48,284 paragraphs. To avoid storing the large Gutenberg database all at once, the dataset is streamed from Hugging Face to the model line by line using a text streamer class built from the Datasets library. Within the streamer class, Regex is used to return tokens that have been stripped of whitespace and punctuation and converted to lowercase.

3 Individual Contribution, Principal Tools, & Languages Used

The English Gutenberg [1] database was collected by Richard Nagyfi (sedthh) on Hugging Face [2]. Word2Vec was developed as part of the Gensim library by Radim Rehurek and others [3]. The dataset streaming library, Datasets, was developed by Hugging Face and others [4].

The data streamer class, training engine, and command line interface were built by me using Python.

4 Testing, Material Limitations, & Dependencies

Many methods of tokenization were tried before settling on Regex expressions, including the built-in Python `.split()` method and the nltk [5] `.word_tokenize()` method. While nltk's tokenizer created a more accurate model on the same number of paragraph blocks, it was significantly slower than using `.split()`. Using Regex expressions achieved a better accuracy than simply using `.split()` while being significantly faster than nltk's tokenizer. Thus, a tokenizer using Regex expressions was chosen to train the final model.

To evaluate accuracy between tokenizers, each tokenizer was used to create a preliminary model that only involved the first 100 paragraphs from the dataset. Then, the model was run to return the five most similar words to words such as "gloomy," "joyous," and "say." The most accurate tokenizer was determined by my satisfaction at the resulting synonyms.

For the final models, the maximum paragraph block size was expanded to the full size of the testing set. In this expansion, my computer ran out of RAM to handle the model training. To resolve this, I had to implement the streamer class to continuously feed tokens to the model as it trained instead of pulling the training set from Hugging Face all at once.

As a visual aid, the training program also implements a TQDM [6] progress bar to visualize how much of the training set the model has received per epoch.

Availability of data and software code

Software code: <https://github.com/virixot/nlp-thesaurus>. Trained model:

References

- [1] Project gutenberg. <https://www.gutenberg.org/>. URL visited on 25th September 2026.
- [2] Richard Nagyfi. Dataset card for project gutenber - english language ebooks. https://huggingface.co/datasets/sedthh/gutenberg_english/, 2023. URL visited on 25th September 2026.
- [3] Radim Rehurek. Gensim: Topic modelling for humans. <https://radimrehurek.com/gensim/>, 2010. URL visited on 25th September 2026.
- [4] Quentin Lhoest Hugging Face et al. Datasets: A community library for natural language processing. <https://github.com/huggingface/datasets>, 2021. URL visited on 25th September 2026.
- [5] Steven Bird, Edward Loper, and Ewan Klein. Natural language toolkit (nltk). <https://nltk.org>, 2009. URL visited on 25th September 2026.
- [6] others Casper da Costa-Luis, Stephen Karl Larroque. Python cli tools. <https://github.com/tqdm/tqdm>, 2019. URL visited on 25th September 2026.